

TECH

REACT CORE

RENDERING HOOKS EFFECTS REACT 18+ / CONCURRENCY

RENDERING

20 ВОПРОСОВ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК react.dev

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 СРЕДНИЙ	ФАЗЫ Q1 · Q2 · Q3	RENDER vs COMMIT	PURE	Как React делит работу на render и commit, почему render обязан быть pure.
ДЕНЬ 2 СРЕДНИЙ	ЧТО ВЫЗЫВАЕТ RE-RENDER Q4 · Q5 · Q6			Side effects в render, причины ререндера, цепочка parent → child.
ДЕНЬ 3 ТЯЖЁЛЫЙ	RECONCILIATION Q7 · Q8 · Q9	KEYS	INDEX-AS-KEY	Как React сравнивает деревья и зачем настоящие keys.
ДЕНЬ 4 СРЕДНИЙ	CONTROLLED vs UNCONTROLLED INPUTS Q10 · Q11 · Q12			Два способа управлять формой и где какой брать.
ДЕНЬ 5 ЛЁГКИЙ	BATCHING Q13 · Q14	AUTOMATIC BATCHING		Как React группирует setState в один рендер.
ДЕНЬ 6 ТЯЖЁЛЫЙ	CONCURRENT RENDERING Q15 · Q16 · Q17			Что такое concurrent режим и зачем прерывание рендера.
ДЕНЬ 7 СРЕДНИЙ	HYDRATION ERS Q18 · Q19 · Q20	MISMATCH	DEBUG RE-RENDER	SSR-гидратация, типичные mismatch и охота на лишние рендеры.

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ

ДЕНЬ 7 — БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1

НАГРУЗКА

СРЕДНИЙ

3 ВОПРОСА

ФАЗЫ

RENDER vs COMMIT

PURE

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Как React делит работу на render и commit, почему render обязан быть pure.

ВОПРОСЫ ДНЯ

Q01 Что такое render phase?

Q02 Что такое commit phase?

Q03 Почему render должен быть pure?

Что такое render phase?

ОПИСАНИЕ

Render phase – первый этап рендеринга в React. Здесь React вызывает компонент-функцию, получает JSX, строит виртуальное дерево (Fiber-узлы), сравнивает с предыдущим и решает, что поменять в DOM. Render phase ДОЛЖЕН быть pure – без побочных эффектов: React может его прервать, отложить, повторить или выкинуть, особенно в concurrent-режиме.

КАК РАБОТАЕТ

- 01 React вызывает функцию компонента, чтобы получить JSX-описание UI.
- 02 Сравнивает новое дерево со старым (reconciliation, см. Q7).
- 03 Записывает список изменений (effect list) – что поменять в DOM на следующей фазе.
- 04 В concurrent – render может быть прерван и перезапущен; поэтому никаких side effects.
- 05 В Strict Mode (dev) функция компонента может быть вызвана ДВАЖДЫ – для отлова не-pure кода.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Каждый render компонента – это вход в render phase.
- 02 useState/useReducer изменения вызывают новую render phase.
- 03 Контекст provider value меняется → потомки идут на render.

ПОДВОДНЫЕ КАМНИ

- 01 Side effects в теле компонента (запрос, мутация, console.log в проде) – антипаттерн. Нужно useEffect.
- 02 Чтение/запись refs во время render – обычно ок для чтения, но запись ломает контракт pure.
- 03 В Strict Mode эффекты двойного вызова режут не-pure код.

КОД

// Pure render

```
function Counter({ count }: { count: number }) {  
  // только подсчёт + JSX, без side effects  
  return <div>{count * 2}</div>;  
}
```

// Антипаттерн – side effect в render

```
function BadCounter({ id }: { id: string }) {  
  fetch(`/api/${id}`); // отправляется при каждом рендере  
  return <div />;  
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем render phase отличается от commit phase?
- 02 Почему React может прервать render?
- 03 Что увидит Strict Mode в нечистом render?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/render-and-commit>

Что такое commit phase?

ОПИСАНИЕ

Commit phase – этап после render, на котором React реально применяет вычисленные изменения к DOM, обновляет refs и запускает effects. В отличие от render, commit phase **СИНХРОННА** и не прерывается. Здесь происходят: insert/update/delete DOM-узлов, `useLayoutEffect` (синхронно после mutation), `useEffect` (асинхронно после paint).

КАК РАБОТАЕТ

- 01 Применить мутации к DOM согласно effect list.
- 02 Обновить refs (привязка/обнуление).
- 03 Запустить `useLayoutEffect` – синхронно, до того как браузер нарисует кадр.
- 04 Браузер делает paint.
- 05 Запустить `useEffect` – асинхронно после paint.
- 06 Cleanup-функции эффектов запускаются перед новой версией эффекта.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любое изменение видимого UI происходит в commit.
- 02 Любая работа с DOM-нодами через ref доступна в commit (`useLayoutEffect/useEffect`).
- 03 Анимации, фокус-менеджмент, измерения – `useLayoutEffect`.

ПОДВОДНЫЕ КАМНИ

- 01 Тяжёлая работа в `useLayoutEffect` блокирует кадр.
- 02 Чтение DOM в `useEffect` – поздно для синхронного позиционирования.
- 03 Двойной commit в Strict Mode (dev) ловит неидемпотентные effects.

КОД

// `useLayoutEffect` – измерение до paint

```
useLayoutEffect(() => {
  const w = ref.current!.offsetWidth;
  setMeasured(w);
}, []);
```

МОГУТ ДОСПРОСИТЬ

- 01 Почему `useLayoutEffect` синхронный, а `useEffect` – нет?
- 02 Что выполняется первым: ref-обновление или `useLayoutEffect`?
- 03 Можно ли прервать commit?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/render-and-commit>

Почему render должен быть pure?

ОПИСАНИЕ

Pure-функция — для одинаковых входов даёт одинаковые выходы и не имеет побочных эффектов. React-компонент должен быть таким же. Причины: React может вызвать его несколько раз (reconciliation, concurrent), отложить или отменить вызов; кэшировать результат (useMemo, React Compiler); запустить дважды в Strict Mode для отладки. Не-pure render ломает все эти оптимизации.

КАК РАБОТАЕТ

- 01 Same inputs (props + state + context) → same JSX output.
- 02 No side effects: запрос, мутация состояния модулей, запись в localStorage — только в effect или event-handler.
- 03 Чтение из refs — допустимо, но осторожно; запись — нет.
- 04 Math.random() и Date.now() в render — нарушение pure: при двойном вызове в Strict дадут разные результаты.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой компонент: pure-render — обязательное правило.
- 02 useMemo и useCallback ожидают pure-логику.
- 03 React Compiler оптимизирует только pure-код.

ПОДВОДНЫЕ КАМНИ

- 01 Запрос к API внутри render — каждый render новый запрос, бесконечный цикл.
- 02 Глобальная мутация (window.x = ...) — ломает SSR и concurrent.
- 03 Math.random() для key — стабильность теряется.

КОД

// Не-pure

```
function Bad() {  
  const id = Math.random(); // меняется каждый рендер  
  return <div data-id={id} />;  
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Что считается side effect в render?
- 02 Почему Math.random в render — антипаттерн?
- 03 Связано ли это с React Compiler?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/keeping-components-pure>

ЧТО ВЫЗЫВАЕТ RE-RENDER

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Side effects в render, причины ререндера, цепочка parent → child.

ВОПРОСЫ ДНЯ

- Q04 Что может произойти, если делать side effects во время render?
- Q05 Что вызывает re-render компонента?
- Q06 Почему parent re-render может вызвать child re-render?

ВОПРОС Q04

Что может произойти, если делать side effects во время render?

ОПИСАНИЕ

Side effects в render – главный источник тонких багов. В лучшем случае – лишние запросы и тормоза; в худшем – бесконечные циклы, рассинхронизация UI/state, мерцания. В concurrent-режиме React может вызывать render несколько раз, отбрасывать промежуточные результаты – все side effects будут повторяться без cleanup.

КАК РАБОТАЕТ

- 01 Запрос в render → новый запрос на каждый render → стек запросов / 429.
- 02 setState в render без условия → бесконечный re-render.
- 03 Запись в module variable → SSR-mismatch (другое значение на сервере и клиенте).
- 04 Мутация props/state – render теряет смысл сравнения.
- 05 Нет cleanup → утечки и race conditions.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Аудит legacy-кода: где fetch/timeout/storage – внутри тела компонента?
- 02 Strict Mode dev – ловит часть таких багов через двойной вызов.
- 03 Профилирование – лишние запросы видны в Network.

ПОДВОДНЫЕ КАМНИ

- 01 setState в render допустим, если условный (например, deriving state from prop) – но это редкий паттерн.
- 02 console.log в render – формально side effect, но не опасный. В проде убирать.
- 03 Concurrent + двойной render = двойной side effect (в dev).

КОД

// Бесконечный цикл

```
function Bad() {  
  const [n, setN] = useState(0);  
  setN(n + 1); // безусловный вызов в render → loop  
  return <div />;  
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие side effects опаснее всего в render?
- 02 Что покажет Strict Mode на этом коде?
- 03 Когда setState в render – допустимо?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/keeping-components-pure>

ВОПРОС Q05

Что вызывает re-render компонента?

ОПИСАНИЕ

Компонент перерисовывается при четырёх причинах: изменился его state (useState/useReducer); изменились его props; перерисовался родитель; изменился context, на который он подписан. Других триггеров нет – useRef.current не вызывает re-render, и мутация state-объекта тоже (нет новой ссылки).

КАК РАБОТАЕТ

- 01 useState setter с НОВЫМ значением (Object.is-сравнение) → rerender.
- 02 useReducer dispatch → rerender, если новый state ≠ старый.
- 03 Parent rerender → React пересчитает поддерево (с учётом мемо).
- 04 Context provider value меняется → все потребители перерисуются.
- 05 useRef.current = ... – НЕ вызывает rerender.
- 06 Мутация state без новой ссылки – НЕ вызывает rerender.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Дебаг лишних re-renderов: разобраться, кто триггерит.
- 02 Оптимизация: убрать context подписку, использовать selector, мемоизировать props.
- 03 React DevTools Profiler – кто и почему рендерился.

ПОДВОДНЫЕ КАМНИ

- 01 Object.is-сравнение state: setN(n) с тем же значением → пропуск ререндера, иногда неожиданно.
- 02 Inline {} / [] / () => в JSX → новая ссылка → React.мемо не сработает.
- 03 Context provider: любая смена value перерисует всех потребителей.

КОД

// Бесполезный setState

```
setN(0); // если n уже 0 – re-render не произойдёт
```

// Inline-объект ломает мемоизацию

```
<C data={{ a: 1 }} /> // каждый рендер новая ссылка
```

МОГУТ ДОСПРОСИТЬ

- 01 Что не вызывает re-render?
- 02 Почему useRef.current не триггерит rerender?
- 03 Как мемоизировать context value?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/render-and-commit#step-2-react-renders-your-components>

ВОПРОС Q06

Почему parent re-render может вызвать child re-render?

ОПИСАНИЕ

По умолчанию React при ререндере родителя вызывает функции всех его потомков заново — без сравнения props. Это оптимально по умолчанию (React не знает, какие props «важные»), но может быть избыточно. Лечение — React.memo (оборачивает компонент, рендер пропускается при равных props), useMemo для дорогих JSX, мемоизация ссылок на функции и объекты.

КАК РАБОТАЕТ

- 01 Parent rerender → React вызывает рендер всех потомков, даже если их props не менялись.
- 02 React.memo — обёртка, делает shallow compare props и пропускает рендер при равных.
- 03 useMemo на JSX-фрагмент — кеширует поддерево.
- 04 Мемо-функции (useCallback) и мемо-объекты (useMemo) — стабильные ссылки в props.
- 05 Ключи (key) разрушают идентичность компонента — при смене key потомок монтируется заново.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Большое поддерево с тяжёлыми компонентами — оборачивать в React.memo.
- 02 Часто рендерящийся parent (например, при наборе текста) — стабилизировать props потомков.
- 03 Карточки списков — React.memo + стабильные key.

ПОДВОДНЫЕ КАМНИ

- 01 React.memo с inline-пропсами не работает — нужно стабилизировать.
- 02 Преждевременная мемоизация дороже наивного render для лёгких компонентов.
- 03 React.memo сравнивает по shallow — глубокие изменения props не уловит.
- 04 С Children-prop React.memo обычно бесполезен.

КОД

```
// React.memo + useCallback
```

```
const Heavy = React.memo(function Heavy({ onClick }) {  
  return <button onClick={onClick}>X</button>;  
});  
const onClick = useCallback(() => doSth(), []);  
<Heavy onClick={onClick} />;
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда React.memo бесполезен?

- 02 Чем отличается React.memo от useMemo?
- 03 Зачем стабилизировать ссылку у callback?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/memo>

ДЕНЬ 3

НАГРУЗКА

ТЯЖЁЛЫЙ

3 ВОПРОСА

RECONCILIATION**KEYS****INDEX-AS-KEY****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Как React сравнивает деревья и зачем настоящие keys.

ВОПРОСЫ ДНЯ

Q07 Как работает reconciliation?

Q08 Зачем нужны keys?

Q09 Почему index как key может быть проблемой?

Как работает reconciliation?

ОПИСАНИЕ

Reconciliation – алгоритм сопоставления старого и нового виртуальных деревьев и вычисления минимального набора изменений в реальном DOM. React обходит деревья, сравнивая позицию и тип узлов: одинаковый тип – переиспользуем DOM-узел и обновляем атрибуты; разный тип – уничтожаем поддерево и создаём заново. Для переупорядочивания списков используются keys.

КАК РАБОТАЕТ

- 01 Один корневой узел – один Fiber-узел, дерево mirror-ится на DOM.
- 02 Сопоставление по позиции и типу: `<div>` с `<div>` – обновление; `<div>` с `<span>` – `replace`.
- 03 Списки с `key` – точное сопоставление по `key`, минимизирует операции.
- 04 При смене `key` компонент полностью размонтируется – `state` сбрасывается.
- 05 Алгоритм линейный по размеру дерева ($O(n)$).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой `render` – `reconciliation` в основе.
- 02 Списки – нужны `keys` для корректного сравнения.
- 03 Принудительный сброс `state` – поменять `key`.

ПОДВОДНЫЕ КАМНИ

- 01 Без `key` или с `index-key` – массив меняется → лишние перемонтирования.
- 02 Условный рендер двух разных компонентов в одной позиции – оба теряют `state`.
- 03 `fragments` не имеют DOM-узла, но имеют позицию в `reconciliation`.
- 04 Перенос узла между разными родителями – это `unmount/mount`, не «`move`».

КОД

```
// Сброс через key
<Profile key={userId} userId={userId} />
// смена userId → unmount + mount, чистый state
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое Fiber и при чём он тут?
- 02 Чем отличается «`move`» от «`remount`»?
- 03 Когда стоит сменить `key` намеренно?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/preserving-and-resetting-state>

Зачем нужны keys?

ОПИСАНИЕ

Keys нужны для стабильной идентификации элементов списка при reconciliation. Без них React сравнивает по позиции – и при вставке элемента в начало все следующие перемонтируются. Хороший key – стабильный, уникальный идентификатор записи (id из БД). Index – допустим только для статичных списков, которые не сортируются и не меняются.

КАК РАБОТАЕТ

- 01 key – атрибут на JSX-элементе списка.
- 02 При reconciliation элементы сопоставляются по key, не по позиции.
- 03 Стабильный key переживает сортировку, фильтрацию, вставки.
- 04 Смена key = unmount + mount, state сбрасывается.
- 05 key должен быть уникален среди siblings, не глобально.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой list-render через .map.
- 02 Принудительный сброс state – сменить key.
- 03 Fragment-keys для группировки строк/ячеек таблицы.

ПОДВОДНЫЕ КАМНИ

- 01 key={index} – баг при сортировке/вставке.
- 02 key=Math.random() – каждый рендер новый key, всё размонтируется.
- 03 Дублирующиеся keys – warning + неопределённое поведение.
- 04 Нет key – warning от React, но рендер всё равно идёт.

КОД

// Стабильный key

```
{users.map(u => <Row key={u.id} user={u} />)}
```

МОГУТ ДОСПРОСИТЬ

- 01 Где key должен быть уникален?
- 02 Что произойдёт при дубликате key?
- 03 Можно ли использовать composite-key?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/rendering-lists>

ВОПРОС 009

Почему index как key может быть проблемой?

ОПИСАНИЕ

index привязан к позиции, а не к содержимому. Если массив сортируется, фильтруется или появляется/исчезает элемент, позиции «сдвигаются», и React сопоставляет компоненты по неправильным элементам. Это приводит к перемешанному локальному state (input-значения, выделения, анимации) и лишней работе DOM. index ок только для статичных, никогда не меняющихся списков.

КАК РАБОТАЕТ

- 01 При вставке в начало: новый элемент – index 0, старый – 1, и т.д. React видит «новый» компонент по index 0 и «обновляет» старый компонент с новыми props.
- 02 input.value, scroll, selection – теряют связь с реальным элементом.
- 03 Анимации воспринимают «обновление» вместо «вставки».
- 04 Сортировка – все элементы получают новые indexes, массив реально не «двигается».

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Только для статичных списков (хедер навигации, константные опции).
- 02 Когда нет id и список не меняется – допустимо.
- 03 В тестах для простых mock-данных.

ПОДВОДНЫЕ КАМНИ

- 01 Самый частый баг с input – вы ввели значение, потом удалили предыдущий элемент → ваш ввод оказался в соседней строке.
- 02 ESLint-плагин react/no-array-index-key.
- 03 Серверная нумерация (id) ≠ локальный index – проектируйте под id.

КОД

// Проблема

```
{items.map((it, i) => (  
  <input key={i} defaultValue={it} />  
))}  
// при удалении первого – input-значения «съезжают»
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда index – допустимый key?
- 02 Какие баги ловит линтер?
- 03 Что делать, если у элементов нет id?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/rendering-lists#why-does-react-need-keys>

ДЕНЬ 4 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

CONTROLLED vs UNCONTROLLED INPUTS

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Два способа управлять формой и где какой брать.

ВОПРОСЫ ДНЯ

- Q10 Что такое controlled component?
- Q11 Что такое uncontrolled component?
- Q12 Когда использовать controlled input?

Что такое controlled component?

ОПИСАНИЕ

Controlled component – форма-элемент (input, select, textarea), у которого значение хранится в React-state. Компонент получает value через props и onChange-обработчик обновляет state. Это даёт полный контроль: валидация, форматирование, derived UI на основе ввода. Минус – ререндер при каждом нажатии.

КАК РАБОТАЕТ

- 01 value={state} + onChange={e => setState(e.target.value)}.
- 02 React контролирует значение элемента – всегда совпадает со state.
- 03 При вводе: keypress → onChange → setState → rerender → новый value в DOM.
- 04 Можно фильтровать ввод, форматировать, валидировать.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Сложная валидация / форматирование (телефон, маска, сумма).
- 02 Реактивный UI (превью введённого).
- 03 Сабмит формы с проверкой обязательных полей.
- 04 В библиотеках react-hook-form, Formik (через register).

ПОДВОДНЫЕ КАМНИ

- 01 Ререндер на каждое нажатие – на больших формах тормозит.
- 02 Нужен дебаунс/throttle для тяжёлых обработчиков.
- 03 Без onChange (только value) – readOnly-режим.
- 04 Несинхронность state и DOM может вызывать каретку-прыжки при некоторых форматированиях.

КОД

// Базовое

```
function Email() {  
  const [v, setV] = useState('');  
  return <input value={v} onChange={e => setV(e.target.value)} />;  
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем хорош/плох контролируемый input для перформанса?
- 02 Как защититься от прыжков каретки?
- 03 Чем react-hook-form отличается?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react-dom/components/input>

Что такое uncontrolled component?

ОПИСАНИЕ

Uncontrolled component – форма-элемент, который сам хранит своё значение в DOM. React читает value через ref при submit или по запросу. Меньше ререндеров, проще интеграция со сторонним кодом, но контроль слабее: нет реактивной валидации на каждое нажатие, derived-UI сложнее.

КАК РАБОТАЕТ

- 01 defaultValue={...} вместо value.
- 02 Нет onChange-state-loop.
- 03 Доступ к значению через inputRef.current.value.
- 04 react-hook-form по умолчанию работает с uncontrolled через ref.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Большие формы – нет лишних ререндеров.
- 02 Интеграция с не-React библиотеками.
- 03 Файловые input (file всегда uncontrolled).
- 04 Прототипы / простые формы submit-only.

ПОДВОДНЫЕ КАМНИ

- 01 Сложно делать реактивное UI (превью).
- 02 Сброс формы – нужно reset через ref.
- 03 Тестирование сложнее – нужно работать с DOM напрямую.
- 04 В Strict Mode dev двойной mount может сбить state, хранимый в ref.

КОД

// Базовое

```
function Form() {
  const ref = useRef<HTMLInputElement>(null);
  return (
    <form onSubmit={e => {
      e.preventDefault();
      console.log(ref.current?.value);
    }}>
      <input ref={ref} defaultValue="" />
    </form>
  );
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда uncontrolled выгоднее?
- 02 Почему file-input всегда uncontrolled?
- 03 Как сбрасывать uncontrolled-форму?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react-dom/components/input>

Когда использовать controlled input?

ОПИСАНИЕ

Controlled – когда нужен контроль над значением: валидация в реальном времени, форматирование (маски, trim, нормализация), derived UI (счётчик символов, подсветка ошибок), синхронизация с другим state. Если нет таких требований – uncontrolled может быть проще и быстрее.

КАК РАБОТАЕТ

- 01 Реактивная валидация: state → derived isValid → UI.
- 02 Форматирование (например, маска телефона) на каждом вводе.
- 03 Синхронизация двух полей (например, total ↔ percent).
- 04 Live-search: state → debounce → fetch.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Live-валидация и подсветка ошибок.
- 02 Динамические маски (телефон, дата, карта).
- 03 Live-search / autocomplete.
- 04 Связанные поля формы (от/до, total/percent).

ПОДВОДНЫЕ КАМНИ

- 01 Все рендеры идут через React – нагрузка на больших формах.
- 02 Реактивная валидация на каждое нажатие – UX иногда плохой (ошибки до конца ввода).
- 03 Каретка-прыжки при ручном форматировании – нужно сохранять/восстанавливать selection.

КОД

// Live-validation

```
const [email, setEmail] = useState('');
const isValid = /\./.test(email);

```

МОГУТ ДОСПРОСИТЬ

- 01 Что лучше для большой формы?
- 02 Когда смешивать controlled и uncontrolled?
- 03 Как избежать каретки-прыжков?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/sharing-state-between-components>

BATCHING AUTOMATIC BATCHING

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Как React группирует `setState` в один рендер.

ВОПРОСЫ ДНЯ

Q13 Что такое `batching`?

Q14 Как работает `automatic batching`?

ВОПРОС Q13

Что такое batching?

ОПИСАНИЕ

Batching – объединение нескольких `setState` в один ререндер. Если в обработчике вызвать `setA`, `setB`, `setC` – React сделает один `render` с обновлёнными значениями. До React 18 batching работал только в обработчиках событий; в 18+ – automatic batching: внутри Promises, `setTimeout`, native event handlers.

КАК РАБОТАЕТ

- 01 Обработчик собирает все `setState`-вызовы.
- 02 После выхода из обработчика React применяет их одной пачкой и делает один `render`.
- 03 `Object.is`-сравнение определяет, нужен ли реальный рендер.
- 04 До 18: только в `SyntheticEvent`. В 18+: везде по умолчанию.
- 05 `flushSync` – принудительный синхронный `flush` (например, для интеграции с DOM-измерениями).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Несколько связанных `setState` в одном обработчике.
- 02 Сетевой колбэк (`then`) с двумя `setState` – в 18+ тоже batched.
- 03 При интеграции с не-React кодом – иногда нужен `flushSync`.

ПОДВОДНЫЕ КАМНИ

- 01 До 18 `setState` в `setTimeout` не batch'ились – два ререндера.
- 02 `flushSync` ломает оптимизации – использовать ограниченно.
- 03 Несколько `setState` на одном объекте без функционального апдейта – затирают друг друга.

КОД

// Batched

```
function onClick() {  
  setA(1);  
  setB(2);  
  setC(3);  
  // один render, не три  
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Что нового в automatic batching React 18?
- 02 Когда нужен `flushSync`?
- 03 Что произойдёт без functional update в цепочке `setN`?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useState#ive-tried-setting-state-but-the-screen-doesnt-update>

ВОПРОС Q14

Как работает automatic batching?

ОПИСАНИЕ

В React 18+ batching включён по умолчанию для всех обновлений: SyntheticEvent, native event, Promise.then, setTimeout, setInterval. Раньше работал только в SyntheticEvent – поэтому await fetch + setA + setB давали два рендера. Теперь – один. Если нужен синхронный промежуточный рендер – flushSync.

КАК РАБОТАЕТ

- 01 React накапливает setState в очереди до конца микрозадачи / тика.
- 02 По завершении – один render с финальными значениями.
- 03 Работает в любом контексте: async/await, then, setTimeout, postMessage.
- 04 Strict Mode не отключает batching.
- 05 flushSync(fn) – внутри fn все setState применяются синхронно, ререндер случится сразу.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой setState – batching работает прозрачно.
- 02 После загрузки данных – set data + set loading в один рендер.
- 03 Интеграция с DOM-измерениями требующими flushSync.

ПОДВОДНЫЕ КАМНИ

- 01 Миграция на 18 – двойные рендеры исчезают, иногда обнаруживаются забытые синхронизации.
- 02 flushSync – снижает преимущества concurrent.
- 03 Тесты со снимком state могут показывать «промежуточный» render иначе.

КОД

// Принудительная синхронность

```
import { flushSync } from 'react-dom';
flushSync(() => {
  setN(5); // render случится тут же
});
domAction(); // DOM уже обновлён
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем automatic batching отличается от старого?
- 02 Когда нужно отключить batching?
- 03 Что меняется при миграции на 18?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/blog/2022/03/29/react-v18#new-feature-automatic-batching>

ДЕНЬ 6 НАГРУЗКА ТЯЖЁЛЫЙ 3 ВОПРОСА

CONCURRENT RENDERING

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Что такое concurrent режим и зачем прерывание рендера.

ВОПРОСЫ ДНЯ

- Q15 Что такое concurrent rendering?
- Q16 Какие проблемы решает concurrent rendering?
- Q17 Почему React может прерывать rendering?

Что такое concurrent rendering?

ОПИСАНИЕ

Concurrent rendering – режим, в котором React может прерывать render, ставить его на паузу, отбрасывать промежуточные результаты и возобновлять, чтобы приоритезировать срочные обновления (ввод, скролл) над медленными (фильтрация большого списка). Включается автоматически при использовании concurrent-API: `useTransition`, `useDeferredValue`, `Suspense`, `createRoot`.

КАК РАБОТАЕТ

- 01 Render – больше не атомарная операция. Может прерываться между Fiber-узлами.
- 02 Срочный апдейт (urgent) – пользовательский ввод, должен идти быстро.
- 03 Несрочный (transition) – фильтр, переключение вкладки. Может ждать.
- 04 React выбирает что рендерить первым по приоритету.
- 05 `createRoot` вместо `ReactDOM.render` – обязательно для concurrent.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 `useTransition` – пометить апдейт как несрочный.
- 02 `useDeferredValue` – отложенное чтение значения для тяжёлого render.
- 03 `Suspense` – показывать fallback пока загрузка.

ПОДВОДНЫЕ КАМНИ

- 01 Side effects в render – двойной запуск, гонки.
- 02 Без `createRoot` concurrent выключен.
- 03 Animation-related код может «дёргаться» при прерываниях.
- 04 Не все библиотеки совместимы – нужен `useSyncExternalStore`.

КОД

```
// createRoot
import { createRoot } from 'react-dom/client';
createRoot(el).render(<App />);
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем `createRoot` отличается от `ReactDOM.render`?
- 02 Какие библиотеки несовместимы с concurrent?
- 03 Что такое tearing?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/blog/2022/03/29/react-v18>

ВОПРОС Q16

Какие проблемы решает concurrent rendering?

ОПИСАНИЕ

Concurrent решает три большие задачи: long-running render блокирует ввод (печать-лаги при фильтре большого списка); приоритезация – срочный ввод должен идти раньше тяжёлого пересчёта; прогрессивный UI – Suspense с fallback вместо застывшего экрана при загрузке. Без concurrent всё это решали через debounce, виртуализацию и ручные оптимизации.

КАК РАБОТАЕТ

- 01 Прерываемый render → срочные апдейты обходят медленные.
- 02 useTransition – программно пометать «не срочно».
- 03 Suspense – declarative loading-state.
- 04 useDeferredValue – отложить чтение значения для медленного потомка.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Live-search по большому датасету.
- 02 Переключение тяжёлой вкладки/маршрута.
- 03 Ленивая подгрузка данных через Suspense + fetch-in-render паттерны.
- 04 Анимации, не зависящие от тяжёлой логики.

ПОДВОДНЫЕ КАМНИ

- 01 Concurrent НЕ ускоряет тяжёлый render сам по себе – только распределяет по приоритету.
- 02 Алгоритмическая медленность ($O(n^2)$) не лечится concurrent.
- 03 Старые библиотеки могут показать tearing.

КОД

// useTransition

```
const [pending, startTransition] = useTransition();
const onChange = (v) => {
  setInput(v); // urgent
  startTransition(() => setQuery(v)); // transition
};
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие проблемы concurrent НЕ решает?
- 02 Когда оно overkill?
- 03 Как concurrent влияет на side effects?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/blog/2022/03/29/react-v18#what-is-concurrent-react>

ВОПРОС Q17

Почему React может прерывать rendering?

ОПИСАНИЕ

Прерывание render – основа concurrent. Если во время тяжёлого render приходит более приоритетный апдейт (пользовательский ввод), React откладывает текущий и берёт срочный. После – может продолжить отложенный или выкинуть, если результаты уже не нужны. Это работает только если render pure – иначе прерванная работа оставит side effects.

КАК РАБОТАЕТ

- 01 Время render порциями (work loop в Fiber).
- 02 В каждой порции React проверяет, есть ли urgent апдейты.
- 03 Если есть – отложить текущий render, начать новый.
- 04 Когда срочный закончится – продолжить отложенный или выкинуть (если устарел).
- 05 Pure-render гарантирует, что отброс не оставит артефактов.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Тяжёлая фильтрация при печати в input.
- 02 Анимации одновременно с пересчётом.
- 03 Suspense, ожидающий данных – fallback вместо застывания.

ПОДВОДНЫЕ КАМНИ

- 01 Side effects в render → двойной запуск или потерянная работа.
- 02 Refs – запись опасна, чтение допустимо.
- 03 Console.log двойной в Strict Mode (dev) – это и есть тренировка для прерываний.

КОД

// Тиричный сценарий

```
// печатаем в input → input меняется быстро
// фильтр (transition) – медленно
// React: input первый, фильтр прерывается
```

МОГУТ ДОСПРОСИТЬ

- 01 Что произойдёт, если в render запустить fetch?
- 02 Как Strict Mode тренирует код к прерываниям?
- 03 Чем pure-render обязателен для concurrent?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/keeping-components-pure>

HYDRATION**MISMATCH****DEBUG RE-RENDERS****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

SSR-гидратация, типичные mismatch и охота на лишние рендеры.

ВОПРОСЫ ДНЯ

Q18 Что такое hydration?

Q19 Какие бывают hydration mismatches?

Q20 Как отлаживать лишние re-render'ы?

ВОПРОС Q18

Что такое hydration?

ОПИСАНИЕ

Hydration – процесс «оживления» серверно-отрендеренного HTML на клиенте. Сервер вернул статическую разметку, браузер её показал; затем React загружается, идёт по тому же дереву компонентов, привязывает обработчики и восстанавливает state. После hydration UI становится интерактивным. Если серверный и клиентский HTML отличаются – hydration mismatch (см. Q19).

КАК РАБОТАЕТ

- 01 `createRoot.hydrateRoot(domNode, <App />)` – client-side hydration.
- 02 React рендерит дерево, ожидая, что DOM уже совпадает.
- 03 Привязывает event handlers к существующим DOM-узлам.
- 04 Восстанавливает state из Suspense-границ.
- 05 В случае несоответствия – выбрасывает warning и перерендеривает целиком (очень дорого).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 SSR (Next.js, Remix, classic Express + ReactDOMServer).
- 02 Static-export (Next export, Astro).
- 03 Streaming SSR с Suspense.

ПОДВОДНЫЕ КАМНИ

- 01 Mismatch – самый частый источник багов SSR.
- 02 Дата/время / `Math.random` / window-зависимый код в `render` → mismatch.
- 03 Динамика на клиенте сразу после hydration может перерисовать всё дерево.
- 04 Большое поддерево в hydration блокирует main thread.

КОД

// Базовый hydrateRoot

```
import { hydrateRoot } from 'react-dom/client';
hydrateRoot(document.getElementById('root')!, <App />);
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем hydration отличается от обычного render?
- 02 Что такое selective hydration?
- 03 Почему mismatch так дорог?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react-dom/client/hydrateRoot>

ВОПРОС Q19

Какие бывают hydration mismatches?

ОПИСАНИЕ

Mismatch – несоответствие серверного и клиентского HTML. Типовые причины: window/document доступ в render (на сервере его нет); Date.now/Math.random; разные локали и часовые пояса; разные feature-flags; разные user-agent-checks; гидрация с разной структурой (например, условный рендер по client-only state). React 18 умеет лучше переносить mismatch через suppressHydrationWarning, но это лекарство, не панацея.

КАК РАБОТАЕТ

- 01 Сравнение серверной и клиентской разметки во время hydration.
- 02 Несоответствие → warning + полный rerender поддерева.
- 03 suppressHydrationWarning – глушит warning для конкретного узла (например, текущей даты).
- 04 Полный rerender теряет состояние ввода и расширяет first paint.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 SSR-проекты – обязательная защита от mismatch.
- 02 Локализованные данные – рендерить через эффект или проксировать.
- 03 Тёмная/светлая тема – определить на сервере или скрыть до hydrate.

ПОДВОДНЫЕ КАМНИ

- 01 typeof window !== 'undefined' в render – антипаттерн под SSR.
- 02 Date.now() в JSX – каждый рендер другая строка.
- 03 Использование localStorage в render → undefined на сервере.
- 04 Фрагменты HTML без id/data-* могут давать неожиданные mismatch.

КОД

// Защита от Date в SSR

```
const [mounted, setMounted] = useState(false);
useEffect(() => setMounted(true), []);
return mounted ? <span>{new Date().toLocaleString()}</span> : null;
```

МОГУТ ДОСПРОСИТЬ

- 01 Что делает suppressHydrationWarning?
- 02 Как обработать тему dark/light без mismatch?
- 03 Чем плох полный rerender при mismatch?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react-dom/client/hydrateRoot#hydrating-server-rendered-html>

ВОПРОС Q20

Как отлаживать лишние re-render'ы?

ОПИСАНИЕ

Стандартный набор: React DevTools Profiler (запись сессии и анализ рендеров с причиной), Highlight Updates в DevTools, why-did-you-render (HOC), console.log с именами компонентов. Цель – найти, КАКОЙ компонент рендерится и ПОЧЕМУ (props, state, context). Лечение – React.memo + стабилизация ссылок (useCallback/useMemo) или вынос context в более узкий scope.

КАК РАБОТАЕТ

- 01 DevTools Profiler: записать сессию, посмотреть flame-график, по компоненту видно «why did this render».
- 02 Highlight Updates подсвечивает каждый рендер в DOM.
- 03 why-did-you-render логирует props-diff в консоль.
- 04 Использовать селекторы (Redux/Zustand) для подписки только на нужное поле.
- 05 Делить context – большой context → много потребителей → много рендеров.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Большая форма / live-search с лагами.
- 02 Список карточек, которые рендерятся при родительском рендере.
- 03 Context, потребители которого мигают.

ПОДВОДНЫЕ КАМНИ

- 01 React.memo не сработает с inline-объектом/функцией в props.
- 02 useCallback с неправильными deps – бесполезен.
- 03 Чрезмерная мемоизация – overhead больше выгоды.
- 04 Profiler в dev замедляет – мерять в production-build.

КОД

```
// Highlight + мемо
// 1) Profiler ищет лишние рендеры
// 2) React.memo + useCallback
const Card = React.memo(({ user, onClick }) => ...);
const onClick = useCallback(() => ..., []);
```

МОГУТ ДОСПРОСИТЬ

- 01 Что лучше: профилировать в dev или в prod?
- 02 Когда мемоизация дороже наивного рендера?
- 03 Как разделить большой context?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/render-and-commit>

ФИНАЛЬНЫЙ ЧЕК - ЛИСТ

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ Развести render и commit phase (Q1, Q2)
- ☐ Объяснить, почему render обязан быть pure (Q3, Q4)
- ☐ Назвать все причины re-render и почему parent тянет child (Q5, Q6)
- ☐ Описать reconciliation и зачем нужны keys (Q7, Q8, Q9)
- ☐ Сравнить controlled и uncontrolled inputs (Q10, Q11, Q12)
- ☐ Объяснить batching и automatic batching (Q13, Q14)
- ☐ Сказать, что такое concurrent rendering и какие проблемы оно решает (Q15, Q16, Q17)
- ☐ Описать hydration и типичные mismatches (Q18, Q19)
- ☐ Назвать инструменты отладки лишних re-renderов (Q20)

EVENT LOOP / ASYNC МЕТОДИЧКА v1
МАТЕРИАЛ ОПИРАЕТСЯ НА learn.javascript.ru